Portfolio

이현수

hyunsuhahaha@gmail.com | 010-2569-7448 | github.com/hyunsuhahaha

NoxGlobal

실무

IT팀 인턴

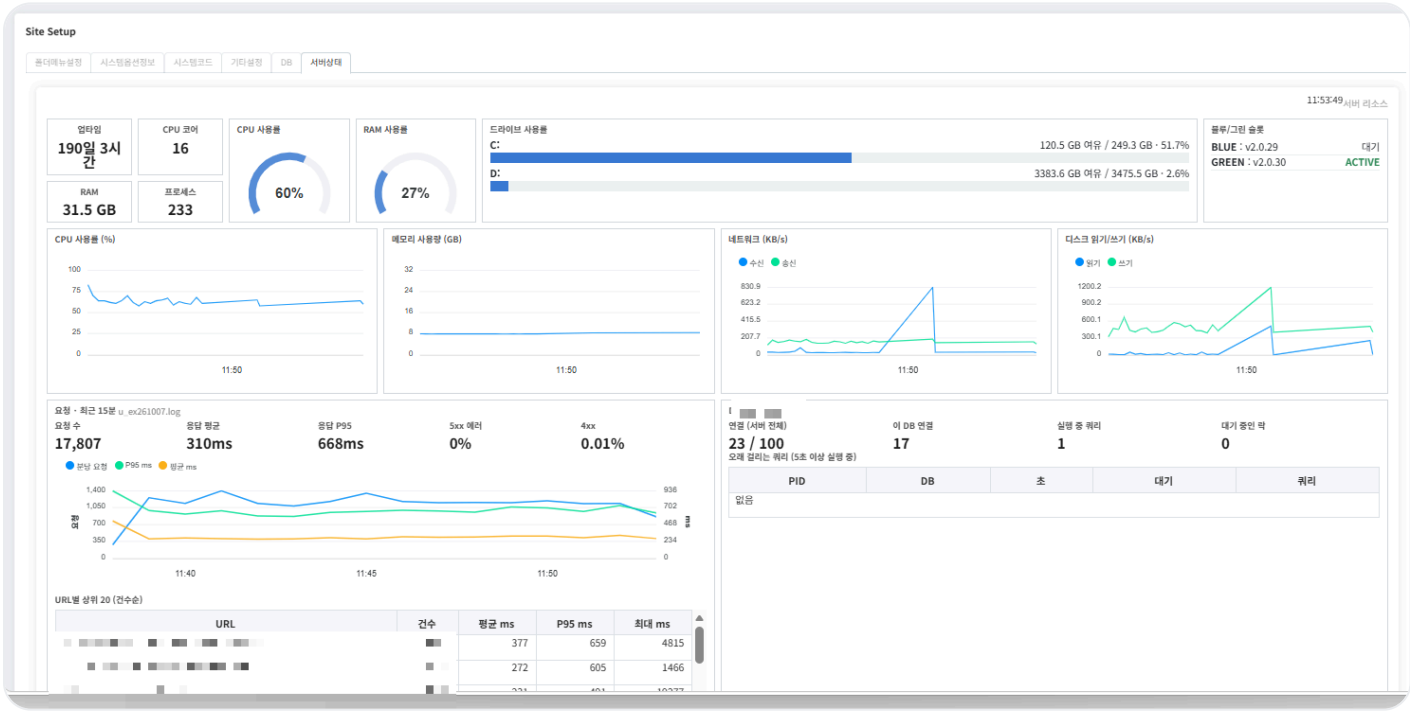
2026.07 – 2026.12 (종료 예정) · IT팀 인턴 · MES 개발 + 배포·운영 인프라

Python · Django 5 | PostgreSQL | MSSQL | IIS · ARR | Gitea Actions | Hyper-V | Airflow | Linux(CentOS) | Apache | MariaDB | Naver Cloud

한 일

- **운영 성능 개선** — 운영 MES 평균 API 응답 **400ms대 → 95ms(약 76%↓)**, CPU 하루 평균 **57% → 30%**.
  - 쿼리 튜닝(N+1 해소, 조회 범위 축소 등)
  - DB 커넥션 재사용(Persistent Connection)으로 요청당 연결 수립 비용 제거
  - Django 템플릿 캐시 로더(cached.Loader) 적용으로 템플릿 처리 시간 최대 86% 단축
  - 동기식 조회 → 비동기 처리로 전환해 사용자 체감 레이턴시 최소화
  - FastCGI 워커 재활용 주기 조정·정적 파일 IIS 직접 서빙으로 워커 부하 감소
- **서버 장애 대응·원인 분석** — 방화벽·리버스 프록시·백엔드 프로세스·DB 서버로 이어지는 요청 흐름과 컴포넌트 간 통신 구조를 분석하고, 계층별 로그 상관 분석으로 장애에 대응.
  - 요청 단위 트레이싱(Django 미들웨어·DB 훅)으로 처리 시간을 DB·외부 DB 연동·애플리케이션 구간으로 분해해 병목·워커 포화 식별
- **MES 관리자 페이지 개선, 서버 모니터링 개발** — 어드민 페이지 역할 기반 접근 제어(RBAC) 적용, 리소스·응답 지연·DB·배포 상태 실시간 모니터링 대시보드 구축.
- **IIS 블루그린 무중단 배포** — IIS 리버스 프록시(ARR)로 BLUE/GREEN 슬롯을 전환하는 무중단 배포 구성.
  - 배포 후 smoke test 실패 시 자동 롤백
  - 배포 시 화면 상태 자동 저장·복원으로 작업 끊김 방지
- **CI/CD 파이프라인, Hyper-V 스테이징 환경 구축·운영** — 온프레미스 Gitea·self-hosted runner로 MES·SCM 등 사내 서비스의 PR 검사·운영 배포·자동 롤백 워크플로를 구성하고, 운영 동일 구성(IIS+PostgreSQL)의 스테이징 환경(Hyper-V)을 구축·운영하며 파이프라인에 연결.
- **회사 홈페이지 CI/CD 구축·운영** — 네이버 클라우드의 CentOS·MariaDB 서버에서 운영하는 회사 홈페이지(noxglobal.com)에 SFTP·SSH 기반 배포 CI/CD 파이프라인 구축·운영.

- **CI 파이프라인 최적화** — PR 검사 시간 **1분대 → 10초대**.
  - 의존성 환경·저장소 캐싱으로 패키지 설치·소스 체크아웃 시간 단축
  - 프로파일링으로 테스트 병목 파악, 준비 단계 캐싱으로 반복 작업 제거
  - 테스트 병렬 실행·중복 실행 취소
- **DB 마이그레이션 자동화** — 스키마·뷰·트리거·프로시저 변경을 테스트 환경에서 migration으로 기록하고, 운영 릴리즈 시 운영 DB에 자동 적용.
- **MES 생산데이터 이상 탐지** — MES·ERP의 투입내역·QR·실자재 데이터에서 누락·불일치를 자동으로 찾는 검사 시스템 개발 (FastAPI·React).
  - MES API·Playwright로 MES·ERP 데이터를 수집해 품목·수량 기준으로 대조
  - Airflow 배치 파이프라인, 멱등성(Idempotency) 보장으로 중복 실행 방지
  - 라인별 누락률과 누락 시점 원본을 대시보드로 제공하고 담당자에게 메일 알림
- **레거시 PHP 서비스 취약점 진단·조치** — SSRF·파일 업로드 취약점을 진단해 URL 화이트리스트·업로드 디렉터리 PHP 실행 차단으로 웹셸·내부망 접근 경로 차단.
- **보안 위협 인텔리전스 기반 정책 개선** — CISA 위협 동향을 분석해 사내 SSL-VPN의 접근통제 위험을 보고하고, 관리자 MFA 적용·접속 IP 제한 등 정책 강화에 기여.
- **현업 커뮤니케이션·요구사항 정리** — 공장 근무자 교육·민원 대응, 현장 요구사항을 문서로 정리해 개발팀에 전달.



SoftBank 해커톤 2026

해커톤 본선 진출

Lily — 클라우드·온프레미스 하이브리드 배포 플랫폼

2026.09 – 10 · 온프레미스·DB·하이브리드 전환 담당 · github.com/SoftBank-team-lily

Java 21 · Spring Boot | k3s · Kaniko | AWS · GCP · RDS | pgroll | Cloudflare Workers · DO · Tunnel

WebSocket Caddy Terraform

고가용성 · 장애 대응

- **장애 자동 전환(failover)** — PC 장애 시 Cloudflare Worker가 연결 실패·엣지 오류·1.5초 무응답을 감지해 요청 단위로 클라우드 대기 Pod(웜 스탠바이)에 재전송. 전환 순간 최대 응답 약 1.6~1.8초로, 예비 수단인 CNAME 전환(유예 60초 + DNS 반영)보다 빠르게 복구. 터널은 붙어 있는데 PC가 멈춘 경우(502)도 장애로 판정.
- **장애 중 읽기·쓰기 보존** — 읽기는 PC 정상 시 저장해 둔 Cache API 사본으로 응답하고, 쓰기(POST)는 Durable Objects 큐에 보관했다가 PC 복구를 확인한 뒤 받은 순서대로 재전송(Idempotency-Key 포함). 장애 직후 첫 요청처럼 판정이 애매한 경우도 재확인 후 큐에 보관.
- **에이전트 복구** — 에이전트가 재시작돼도 앱 슬롯·DB 터널·버스팅·스키마 상태를 다시 붙여 공개 주소가 끊기지 않게 하고, RDS 터널이 끊기면 자동 재연결. 컨트롤 플레인이 잠시 끊겨도 진행 중인 작업이 바로 실패하지 않도록 처리.
- **하이브리드 컨트롤 플레인** — 클라우드와 온프레미스 에이전트를 WebSocket으로 연결해 토큰 인증·명령 중계·DB 터널 인증서 발급을 중앙에서 관리. Cloudflare API는 플랫폼이 에이전트 대신 호출해 에이전트에 클라우드 자격 증명을 두지 않음.

거점 이전 · 확장

- **거점 전환·클라우드 간 이전** — 앱을 PC↔클라우드, AWS↔GCP로 옮길 때 DB를 함께 이전(PC↔RDS는 SSH 리버스 터널, 클라우드 간은 PostgreSQL 복사). 이전 중에는 앱 쓰기를 멈췄다 다시 열어 데이터 정합성을 지키고, DB 없는 앱은 다운타임 없이 이전. 진행 단계 표시·취소와 버스팅과의 동시 실행 차단.
- **멀티클라우드 동시 운영** — 같은 앱을 AWS·GCP에 함께 띄우고 엣지 Worker가 설정 비율대로 요청을 분산. DB는 GCP Cloud SQL 하나에 두고 AWS 쪽은 DB 릴레이로 접속. 한쪽이 502/503/504를 내면 GET은 반대쪽으로 재전송.
- **클라우드 버스팅** — PC 부하가 임계치를 넘으면 연결 단위가 아니라 요청 단위로 초과분만 클라우드로 분산. 클라우드로 넘긴 유휴 keep-alive 연결은 회수하고, 대기 Pod를 유지해 기동 지연 없이 흡수.

DB · CI/CD

- **멀티테넌트 DB 프로비저너** — 공용 RDS에 프로젝트별 DB·계정 생성을 자동화, 테넌트 격리·연결 수 제한. DB 비밀번호는 SSM Parameter Store(SecureString·KMS)에 두고 플랫폼 메타데이터에는 참조 경로(secretRef)만 저장. 워커에 권한을 주면 사용자 앱이 다른 테넌트 비밀번호를 읽을 수 있어, SSM 접근 권한은 컨트롤 노드에만 부여해 노드 단위로 권한 경계를 나눔. Terraform으로 관리.
- **Expand-Contract 패턴 기반 무중단 스키마 마이그레이션(pgroll)** — 구·신 버전 앱이 함께 동작하도록 스키마를 단계적으로 확장·정리하고, 실패 시 앱과 스키마를 함께 롤백. 두 버전 동시 운영을 깨는 연산은 적용 전 거절하고, 카나리 실패 시 시작한 마이그레이션을 즉시 되돌림. 온프레미스(내 PC) 배포에도 동일 적용.
- **빌드 설정 자동 감지·배포** — 레포 파일에서 포트·DB·헬스체크 경로를 자동 감지해 Dockerfile을 생성하고, 백엔드+프론트를 Caddy 뒤 단일 이미지로 번들. GitHub → Kaniko → ECR → 블루그린, 전환 전 30초 카나리로 오류율·p95를 검증. 온프레미스는 20%씩 전환하며 새 슬롯의 5xx가 늘면 되돌림. 배포 실패 시 규칙·AI로 원인을 진단해 설정을 고쳐 최대 3번 재배포.

pamon

개인

외부 API 장애 원인 추적용 SRE 모니터.

2026.10 · 개인 프로젝트 · github.com/hyunsuhahaha/public-api-monitor · pamon.hyunsuhahaha.workers.dev

Go | SQLite · Litestream | GCP | Docker | Caddy | Cloudflare Workers · D1 | GitHub Actions

장애 판정

- 공공 API는 실패해도 HTTP 200을 줌. 응답 본문의 결과 코드까지 읽어서 판정하고, 키 미등록·호출 한도 같은 우리 쪽 문제는 장애에서 뺌.
- 인증이 필요한 API는 키 없이 호출해서 그 서비스가 직접 거절하는지로 확인. CDN이 대신 준 403이나 점검 페이지는 정상으로 안 침.
- API마다 최근 응답으로 평소 속도를 학습해서 느림 기준을 정함.

원인 추적

- 요청을 DNS·TCP·TLS·서버 대기 구간으로 나눠서 어디서 멈췄는지 기록 (Go *httptrace*).
- 타임아웃이 나면 같은 요청을 한 번 더 길게 보내서, 일시적으로 걸린 건지 서버 전체가 느린 건지 구분.
- 같은 시각 다른 서버, 같은 기관 다른 API 결과와 비교해서 기관 서버 문제인지 네트워크 문제인지 근거를 남김.

SLO · 장애 대응

- API별 SLO, 에러 버짓, burn-rate. 장애 기록, 포스트모템 초안, 런북.
- 하룻밤 돌려보니 응답 형식 변경 알림 6건이 다 오탐이었음. 에러 응답은 학습에서 빼고, 느림 기준은 정상 응답으로만 계산하게 고침.

운영

- GCP VM 한 대에서 5분마다 검사하고 결과를 정적 사이트로 내보냄 (systemd 타이머, Docker, Caddy).
- SQLite를 Litestream으로 GCS에 실시간 복제.
- Cloudflare Workers를 공개 입구로 두고 접속 기록은 D1에 저장.
- 서버와 분리된 GitHub Actions에서 5분마다 사이트가 최신인지 확인.

gitea-auto-reviewer

실무 · 오픈소스

온프레미스 Gitea AI 코드 리뷰어

개인 프로젝트 · MIT · github.com/hyunsuhahaha/gitea-auto-reviewer

Python 3.11 (stdlib only) | Codex CLI | GitNexus | Gitea Actions | Windows runner | Django · pytest

리뷰 파이프라인

- **정적 분석 (1차)** — GitNexus 의존성 그래프로 변경 심볼의 호출자·피호출자·영향 경로를 추적하고, Codex가 diff·레포 전체·그래프를 함께 보고 bug·security·performance·dependency·policy 후보를 최대 5건 생성. 모든 지적에 실제 file:line 근거를 요구하고 존재 여부를 검증.
- **재현·검증 (2차)** — 후보를 테스트 DB의 transaction.atomic() 안에서 Django view·ORM을 직접 호출해 재현하고 expected/observed로 판정한 뒤 강제 롤백. 불확실하면 재현 스크립트를 자동 수정해 1회 재실행.
- **분류** — 재현·검증 성공은 "재현된 문제", 미재현은 "재현하지 못한 발견", 재현했지만 반증 검토에서 탈락한 것은 "2차 검증 미채택"으로 나눠 PR 댓글에 게시. 한 번 안 걸렸다고 지적을 삭제하지 않음.

안전 · 설계

- **권한 최소화** — pull_request_target 신뢰 워크플로에서 동일 저장소 PR만 처리하고, PR 코드는 실행하지 않고 base만 가져옴. 리뷰는 읽기 전용 샌드박스, Gitea 토큰은 댓글 단계에서만 사용. 승인·머지·푸시·브랜치 보호 변경은 하지 않는 1차 리뷰어.
- **재현 결과를 head SHA에 바인딩**(evidence.json)해 같은 커밋의 결과를 보존. 런타임 의존성 없이 Python 표준 라이브러리만 사용.

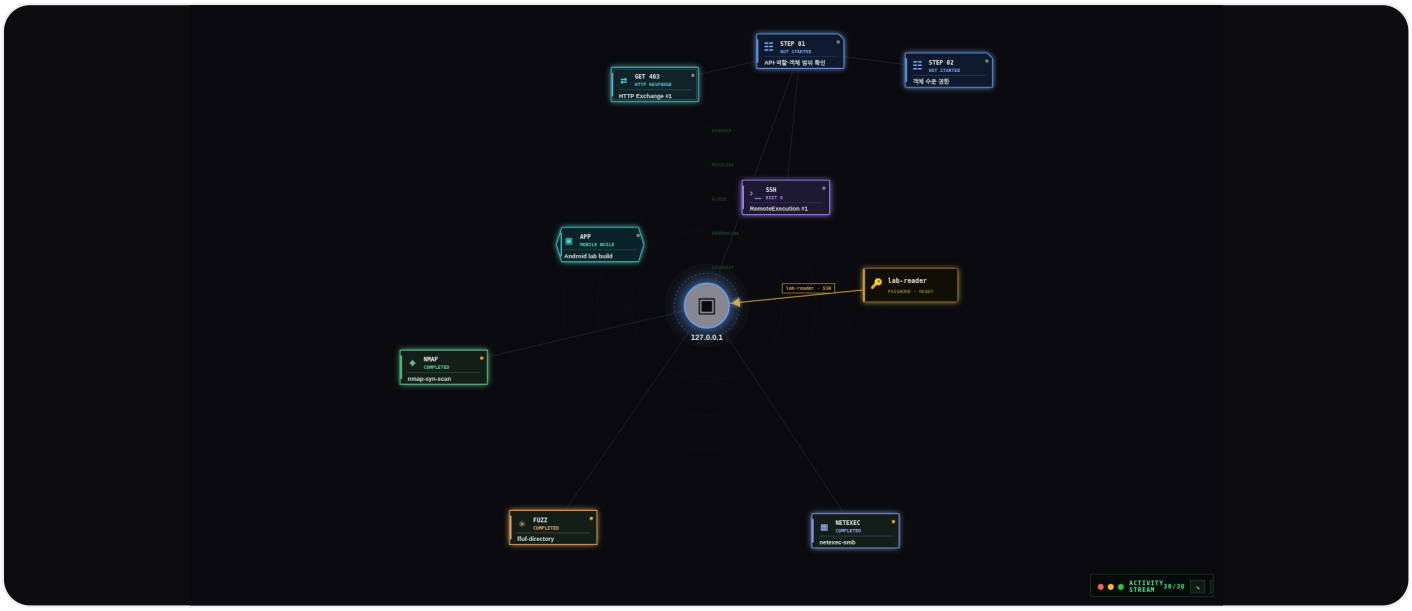
ShadowTrace

개인

통합 펜테스팅·조사 기록 관리 플랫폼

개인 프로젝트 · github.com/hyunsuhahaha/ShadowTrace

Python · FastAPI | eBPF (BCC) | SQLite | React · Vite · sigma.js | Nmap · ffuf · NetExec · Impacket



Progress Graph — 대상·도구 실행·자격증명·점검 단계 연결 (합성 데이터로 촬영)

주요 기능

- **정찰·웹 테스트** — Nmap 서비스 탐색과 feroxbuster·ffuf 기반 디렉토리·가상호스트·파라미터 퍼징 지원. HTTP 요청 편집·페이로드 변형·응답 비교와 프록시 캡처로 웹 동작 검증.
- **인증·AD 점검** — NetExec 기반 프로토콜별 인증 확인과 SMB 공유 탐색 지원. Kerberos 사용자 열거, AS-REP·Kerberoasting, 제한된 위임(S4U2Self/S4U2Proxy) 등 도메인 점검의 실행 결과·증거 관리.
- **PoC 검증·접근 후 조사** — SearchSploit 후보 검색부터 PoC 원본·작업 사본·수정 이력·실행 결과 관리. SSH·WMI·WinRM 원격 작업과 LinPEAS 분석·SUID 점검으로 접근 후 조사 지원.
- **자격증명·해시 관리** — 획득한 계정의 출처를 기록하고 후속 작업에 연결. Hashcat·John 크래킹과 DPAPI·LSASS 분석 등 자격증명 처리 결과를 조사 기록에 통합.

활동 추적·증거 관리

- **eBPF 기반 실행 흐름 복원** — 프로세스·TTY 입출력·소켓·파일 이벤트를 수집하고 프로세스 계보·세션·시각을 기준으로 명령 흐름 재구성. 이벤트 손실·신뢰도를 기록하고 귀속이 불명확한 활동은 원본 보존.
- **조사 경로·보고서 연결** — 대상·실행·자격증명·증거·Finding을 Progress Graph로 연결하고 Runbook으로 점검 진행 관리. 증거 출처·SHA-256과 선택한 경로를 스냅샷으로 보관해 PDF·DOCX 등 보고서로 내보내기.